

CO2070
COMPUTER ARCHITECTURE

E/22/014 , E/22/035

GROUP 06

E/22/014 , E/22/035

LAB 6

SEMESTER 4

08/08/2026

1. HANDLING CACHE MISSES (READ AND WRITE)

Our cache implements the write-back, write-allocate policy. This contrasts with write-through, where every write is propagated to main memory immediately (simpler, but far more memory traffic). With write-back, a modified (dirty) block is written to memory only when it is evicted from the cache — not on every write — which is why a per-line dirty bit is needed to track whether the block differs from memory.

Read miss:

- 1) Tag comparison fails ($\text{hit} = 0$) for the addressed line.
- 2) If the currently-resident line is dirty, it is first written back to main memory (the evicted block's old address = {old tag, index}). If it is clean, the write-back step is skipped and the old data is simply discarded.
- 3) The missing block is then fetched from main memory into the cache line, and the tag, valid and dirty ($=0$) bits are updated.
- 4) The original read request is now replayed against the freshly-installed line, which resolves as an ordinary hit: the requested word is selected by offset and returned to the CPU, and BUSYWAIT is de-asserted.

Write miss:

Handled identically to a read miss for the allocate/fetch part (write-back the evicted dirty block if needed, then fetch the missing block from memory into the line) — this is the write-allocate policy: a store never bypasses the cache, it is always brought in first.

Once the block is installed, the original write is replayed as a hit: the CPU's data word is merged into the block at the correct offset and the dirty bit is set to 1, ready to be written back on a future eviction.

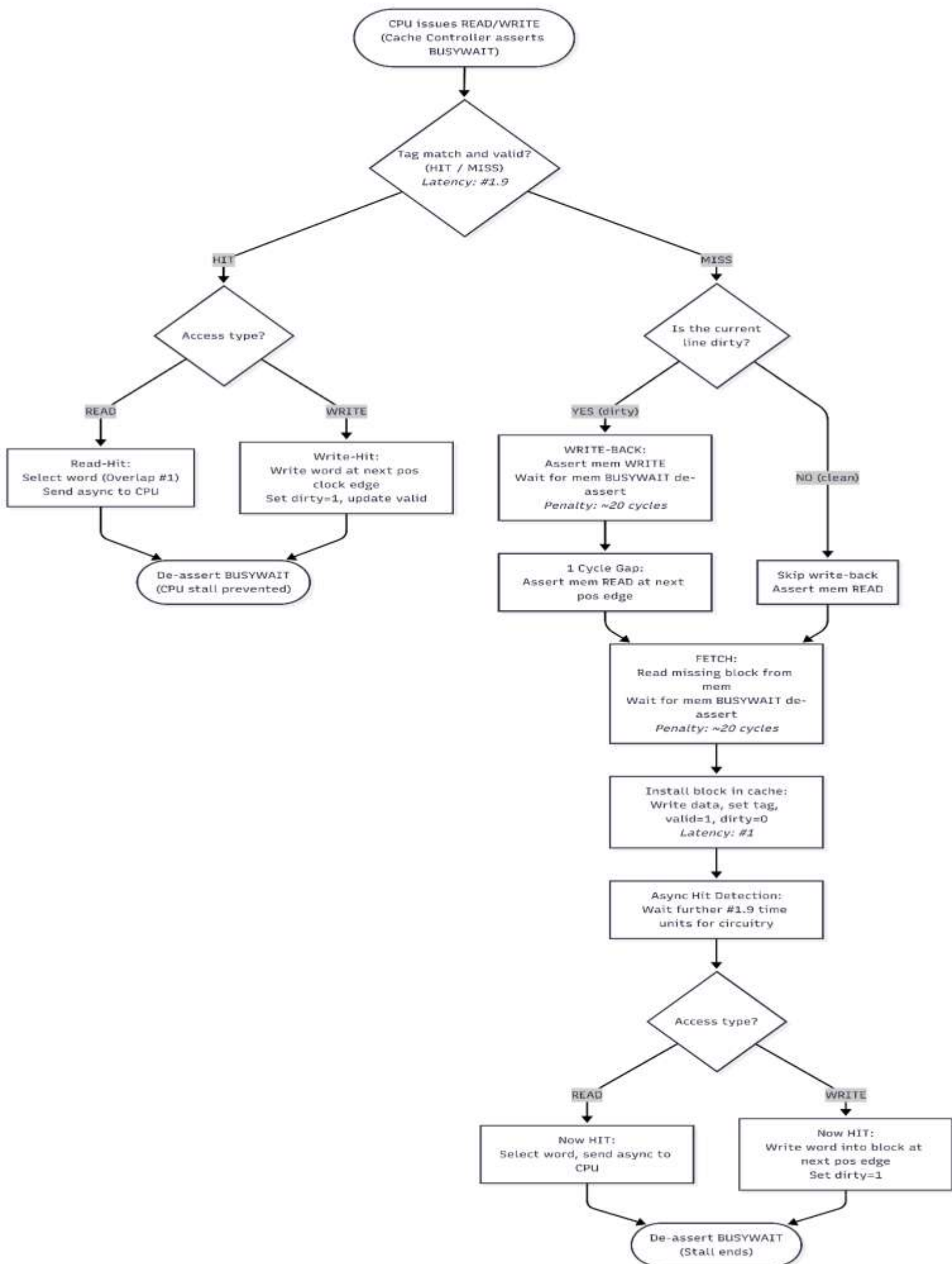


Figure 1: Cache miss handling flow (read-miss and write-miss share the allocate/fetch path; they only differ in what happens once the line becomes a hit)

2. CACHE MAPPING MECHANISM

We implement a direct-mapped cache with 8 lines (sets) and a block size of 4 bytes (1 word = 4 bytes, matching the Lab 6 block-based data memory). Each memory address maps to exactly one cache line, determined by its index bits.

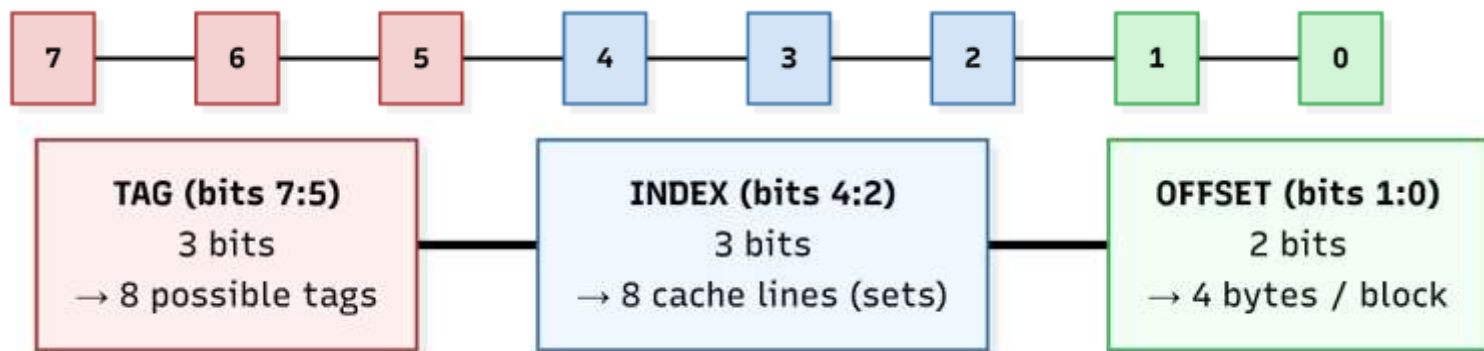


Figure 2: 8-bit CPU-side address format for the direct-mapped cache.

Field	Bits	Width	Purpose
TAG	[7:5]	3 bits	Identifies which memory block currently occupies the line
INDEX	[4:2]	3 bits	Selects 1 of 8 cache lines ($2^3 = 8$ sets)
OFFSET	[1:0]	2 bits	Selects 1 of 4 bytes within the 4-byte block

The memory-side address (to the block-based Lab-6 data memory) is 6 bits = {tag, index}, since it addresses 4-byte blocks rather than individual bytes

3. CACHE TABLE TRACE

Instructions:

loadi 0 0x09 ·
loadi 1 0x01 ·
swd 0 1 ·
swi 1 0x00 ·
lwd 2 1 ·
lwd 3 1 ·
sub 4 0 1 ·
swi 4 0x07 ·
lwi 5 0x07 ·
lwi 6 0x20 ·
swi 4 0x20

Register values as the program runs: r0=0x09, r1=0x01 (from the two loadi's), r2=r3=mem[0x01]=0x09 (from the two lwd's), r4 = r0-r1 = 0x08 (from sub), r5=mem[0x07], r6=mem[0x20]. Addresses used: 0x01, 0x00 (both tag=000,index=000 → line 0), 0x07 (tag=000,index=001 → line 1), 0x20 (tag=001,index=000 → line 0, conflicts with the block already resident there).

#	Instruction	Addr (T:I:O)	Result	Cache action
1	loadi r0,0x09	-	r0=09	no memory access
2	loadi r1,0x01	-	r1=01	no memory access
3	swd r0,(r1)	0x01 = 000:000:01	-	COLD MISS → write-allocate line0 (tag=000); fetch 00000000, merge byte1=09 → data=00000900, dirty=1
4	swi r1,0x00	0x00 = 000:000:00	-	HIT on line0; merge byte0=01 → data=00000901, dirty=1
5	lwd r2,(r1)	0x01 = 000:000:01	r2=09	HIT on line0 (byte1=09)
6	lwd r3,(r1)	0x01 = 000:000:01	r3=09	HIT on line0 (byte1=09)
7	sub r4,r0,r1	-	r4=08	no memory access
8	swi r4,0x07	0x07 = 000:001:11	-	COLD MISS → write-allocate line1 (tag=000); fetch 00000000, merge byte3=08 → data=08000000, dirty=1
9	lwi r5,0x07	0x07 = 000:001:11	r5=08	HIT on line1 (byte3=08)

10	lwi r6,0x20	0x20 = 001:000:00	r6=00	MISS, tag mismatch on line0 (has tag=000, dirty=1) → WRITE-BACK old block (00000901) to mem[0x00-0x03], then FETCH new block (all zero) → line0 becomes tag=001, data=00000000, dirty=0; then HIT, r6=byte0=00
11	swi r4,0x20	0x20 = 001:000:00	-	HIT on line0 (tag=001 now resident); merge byte0=08 → data=00000008, dirty=1

Final cache table (verified by RTL simulation of this exact instruction sequence):

Line	Valid	Dirty	Tag	Data block [byte3 byte2 byte1 byte0]
0	1	1	001	00 00 00 08
1	1	1	000	08 00 00 00
2	0	0	-	-
3	0	0	-	-
4	0	0	-	-
5	0	0	-	-
6	0	0	-	-
7	0	0	-	-

Final register values: r0=0x09, r1=0x01, r2=0x09, r3=0x09, r4=0x08, r5=0x08, r6=0x00. Note line 0 ends up holding tag=001 (the 0x20 block) because it was the second, more recent eviction winner; the original tag=000 block for line 0 was written back to memory at addresses 0x00-0x03 before being replaced.

4. COMPLETED CACHE CONTROLLER FSM

The provided skeleton only showed the IDLE and MEM_READ states, covering the clean-miss (no write-back needed) path. To fully implement the write-back policy, we added the missing MEM_WRITE_BACK state together with two transitions that were absent from the skeleton:

- IDLE → MEM_WRITE_BACK, taken when (read=1 || write=1) && hit=0 && dirty=1 (a miss where the line being evicted is dirty and must be flushed to memory first).
- MEM_WRITE_BACK → MEM_READ, taken once mem_busy=0 (the write-back to memory has completed, so we can proceed to fetch the missing block). MEM_WRITE_BACK also has a mem_busy=1 self-loop identical in spirit to MEM_READ's, since the write-back itself is a multi-cycle memory operation.
- MEM_READ → IDLE, taken once mem_busy=0 (the fetch has completed and the new block has been installed; the originally-stalled CPU request is now re-resolved as an ordinary hit on re-entering IDLE). This return transition was missing from the skeleton — without it the FSM would never leave MEM_READ after a fetch completed.

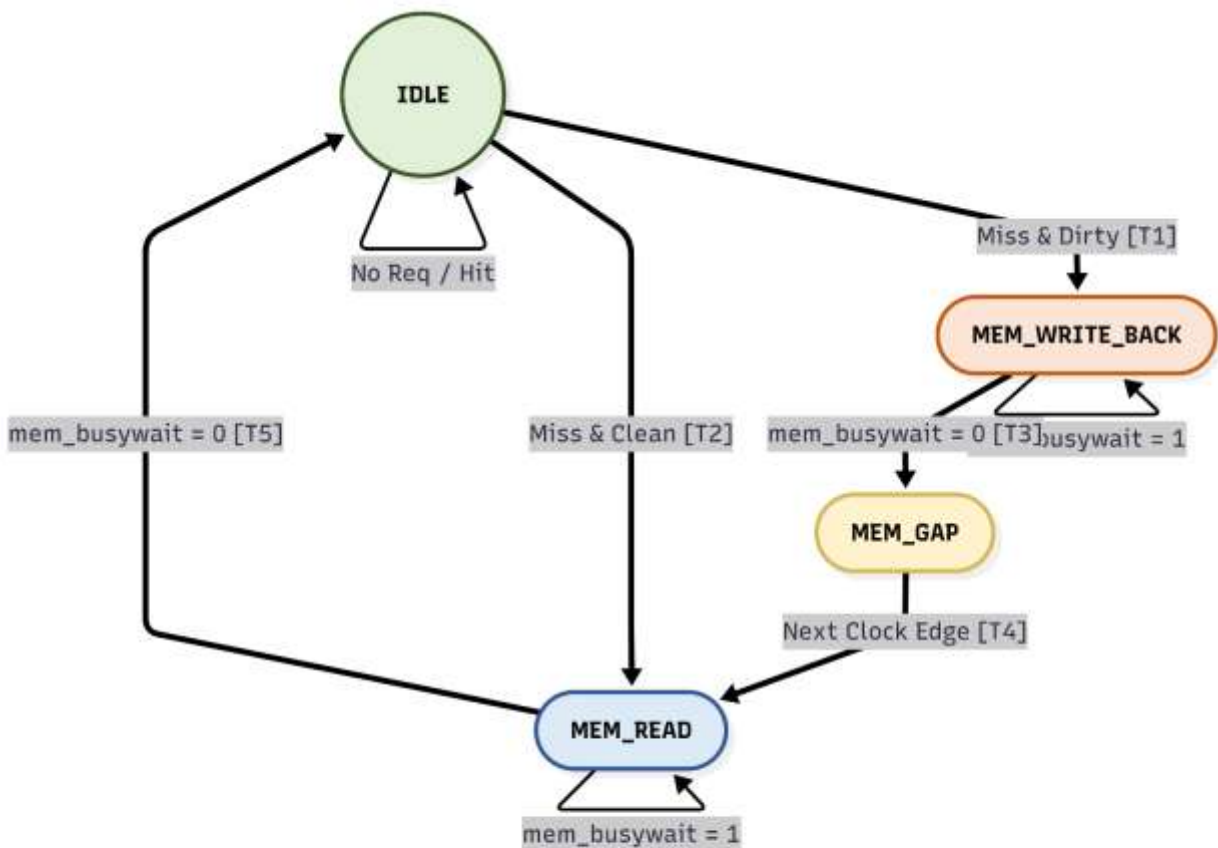


Figure 3: Completed FSM. Black = given in the skeleton. Orange = added to complete the write-back miss path and the missing return-to-IDLE transition

Note: on a clean miss the FSM still goes IDLE to MEM_READ directly (skipping MEM_WRITE_BACK), exactly as in the given skeleton; the added state and transitions only activate the write-back path when the evicted line is dirty, and add the completion edge back to IDLE for both paths